ML Problem Solving: Pytorch with Perceptrons, Adaline, MLP, Optimizers

Overview

This problem-solving develops a deep, ground-up comprehension of machine learning optimization and neural architectures. Many high-level frameworks abstract away matrix calculus, loss surface dynamics, and numerical stability routines. In Problems 1 through 4 of this workshop, you will implement all core algorithms from first principles using pure PyTorch tensors. In Problem 5, you will bridge these tensor implementations with PyTorch's standard optimization library (`torch.optim`) to systematically tune hyperparameters.

Coding Requirements

1. **Pure PyTorch Tensor Operations (Problems 1-4):** All linear transformations, element-wise activations, loss evaluations, backpropagation gradients, and manual parameter updates must be executed using PyTorch (`torch.Tensor`) primitives without relying on `torch.nn` or `torch.autograd.backward()`.
2. **PyTorch Optimizer API (Problem 5):** Problem 5 transitions to using the native PyTorch optimizer classes (`torch.optim.SGD` and `torch.optim.Adam`) to demonstrate production optimization integration with custom tensor models.
3. **Dataset Partitioning Integrity:** Parameters for feature standardization (mean μ and standard deviation σ) must be computed strictly from the training split and reused to normalize test partitions.

Problem Breakdown

Problem	Topic	Target datasets
Problem 1	Manual Data Processing Pipeline	Iris Binary & Wine Multiclass
Problem 2	Adaptive Linear Neuron (Adaline)	Iris Binary Dataset
Problem 3	Classical Rosenblatt Perceptron	Iris Binary Dataset
Problem 4	Two-Layer Perceptron (MLP)	Wine Multiclass Dataset
Problem 5	PyTorch API Optimizers & Grid Search	Wine Multiclass Dataset

Data Specifications and Acquisition

Two benchmark datasets from `sklearn.datasets` are used across the problems. Load and prepare them as follows:

Dataset A: Binary Classification (Iris Setosa vs. Versicolor)

- **Target Allocation:** Problem 1, Problem 2, Problem 3.
- **Source:** `sklearn.datasets.load_iris`
- **Subset:** First 100 samples (Classes 0 and 1).
- **Features Selected:** Petal length (index 2) and sepal length (index 0).
- **Target Encoding:** $y \in \{-1.0, +1.0\}$.
- **Acquisition Code:**

```
from sklearn.datasets import load_iris
import torch

iris = load_iris()
X_iris_raw = iris.data[:100, [0, 2]]  # Sepal length, petal length
y_iris_raw = iris.target[:100]        # 0: Setosa, 1: Versicolor

# Map 0 -> -1.0, 1 -> +1.0
y_iris_raw = [-1.0 if val == 0 else 1.0 for val in y_iris_raw]

X_iris = torch.tensor(X_iris_raw, dtype=torch.float32)
y_iris = torch.tensor(y_iris_raw, dtype=torch.float32)
```

Dataset B: Multi-Class Non-Linear Classification (Wine Dataset)

- **Target Allocation:** Problem 1, Problem 4, Problem 5.
- **Source:** `sklearn.datasets.load_wine`
- **Subset:** All 178 samples, 13 chemical features, 3 target classes ($C = 3$).
- **Target Encoding:** Class index $y \in \{0, 1, 2\}$, converted internally to one-hot vectors $Y \in \{0, 1\}^{N \times 3}$.
- **Acquisition Code:**

```
from sklearn.datasets import load_wine
import torch

wine = load_wine()
X_wine = torch.tensor(wine.data, dtype=torch.float32)
y_wine = torch.tensor(wine.target, dtype=torch.int64)
```

Problem 1: Manual Data Processing Pipeline (Both Datasets)

Associated Concepts & Theory Machine learning algorithms optimized via first-order gradient methods are sensitive to feature variances. Disproportionate feature scales skew the geometry of the optimization objective into ill-conditioned ellipsoidal surfaces, which impedes convergence. Z-score standardization transforms each feature column to have zero sample mean ($\mu = 0$) and unit sample variance ($\sigma^2 = 1$). To avoid data leakage, normalization parameters must be computed strictly on the training partition and applied identically to test partitions.

Mathematical Formulations For a design matrix $X \in \mathbb{R}^{N \times D}$:

$$\mu_j = \frac{1}{N} \sum_{i=1}^{N} X_{i,j}, \quad \sigma_j = \sqrt{\frac{1}{N} \sum_{i=1}^{N} (X_{i,j} - \mu_j)^2 + \epsilon}$$

$$X_{\text{norm},i,j} = \frac{X_{i,j} - \mu_j}{\sigma_j}$$

where $\epsilon = 10^{-7}$ prevents division by zero for features with zero variance.

Tasks Checklist

- ☐ Implement `manual_train_test_split_and_standardize(X, y, train_ratio=0.7, seed=42)`.
- ☐ Partition X and **y** using `torch.randperm(num_samples)` seeded via `torch.manual_seed(seed)`.
- ☐ Calculate column-wise mean μ and standard deviation σ using `torch.mean` and `torch.std` with `keepdim=True` strictly on X_{train}.
- ☐ Normalize both X_{train} and X_{test} using the computed training moments.
- ☐ **Dataset Execution:** Execute this function on **both** Dataset A (Iris Binary) and Dataset B (Wine Multiclass) to generate their respective standardized training and testing splits.

Problem 2: Adaptive Linear Neuron (Adaline) with Batch Gradient Descent (Iris Binary Dataset)

Associated Concepts & Theory The Adaptive Linear Neuron (Adaline) introduces learning through a continuous linear activation function, distinguishing it from discrete step models. Parameters are optimized via Batch Gradient Descent on a differentiable Mean Squared Error (MSE) objective function. Once training converges, classification decisions are produced via a sign threshold quantizer.

Mathematical Formulations

- **Net Input & Linear Activation:**

$$z^{(i)} = \mathbf{w}^T \mathbf{x}^{(i)} + b, \quad \phi(z^{(i)}) = z^{(i)}$$

- **Mean Squared Error Objective Function:**

$$J(\mathbf{w}, b) = \frac{1}{2N} \sum_{i=1}^{N} \left(y^{(i)} - z^{(i)} \right)^2$$

- **Vectorized Analytical Gradients:**

$$\frac{\partial J}{\partial \mathbf{w}} = -\frac{1}{N} X^T (\mathbf{y} - \mathbf{z}), \quad \frac{\partial J}{\partial b} = -\frac{1}{N} \sum_{i=1}^{N} (y^{(i)} - z^{(i)})$$

- **Batch Gradient Descent Parameter Update Rule:**

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{\partial J}{\partial \mathbf{w}}, \quad b \leftarrow b - \eta \frac{\partial J}{\partial b}$$

- **Decision Quantizer:**

$$\hat{y}^{(i)} = \begin{cases} +1.0 & \text{if } z^{(i)} \geq 0.0 \\ -1.0 & \text{if } z^{(i)} < 0.0 \end{cases}$$

Tasks Checklist

- ☐ **Dataset Execution:** Train and evaluate strictly on the **Iris Binary dataset** (Dataset A).
- ☐ Implement class `AdalineGD(learning_rate=0.01, epochs=50, seed=42)`.
- ☐ Initialize weights $\mathbf{w} \in \mathbb{R}^D$ from a normal distribution $\mathcal{N}(0, 0.01^2)$ and bias $b \in \mathbb{R}^1$ to zero using pure PyTorch tensors.
- ☐ Vectorize the forward pass: compute $\mathbf{z} = X\mathbf{w} + b$ using `torch.matmul`.
- ☐ Compute exact analytical gradients $\frac{\partial J}{\partial \mathbf{w}}$ and $\frac{\partial J}{\partial b}$.
- ☐ Update weights and bias simultaneously at each epoch.
- ☐ Record MSE loss at each iteration in `self.cost_history`.
- ☐ Implement `predict(X)` using `torch.where` to return class labels $\{-1.0, +1.0\}$.

Problem 3: Online Rosenblatt Perceptron (Iris Binary Dataset)

Associated Concepts & Theory The classical Rosenblatt Perceptron uses an error-driven online learning algorithm. Unlike Adaline, the Perceptron passes the net linear combination directly through a non-differentiable Heaviside step function during the forward pass. Model weights and biases are updated incrementally after inspecting each individual training example, rotating the hyper-plane only when a prediction error is encountered.

Mathematical Formulations

- **Heaviside Step Activation:**

$$z^{(i)} = \mathbf{w}^T \mathbf{x}^{(i)} + b$$

$$\hat{y}^{(i)} = \begin{cases} +1.0 & \text{if } z^{(i)} \geq 0.0 \\ -1.0 & \text{if } z^{(i)} < 0.0 \end{cases}$$

- **Sample-by-Sample Parameter Update Rule:** For each instance $(\mathbf{x}^{(i)}, y^{(i)})$:

$$\Delta \mathbf{w} = \eta \left(y^{(i)} - \hat{y}^{(i)} \right) \mathbf{x}^{(i)}$$

$$\Delta b = \eta \left(y^{(i)} - \hat{y}^{(i)} \right)$$

$$\mathbf{w} \leftarrow \mathbf{w} + \Delta \mathbf{w}, \quad b \leftarrow b + \Delta b$$

Tasks Checklist

- ☐ **Dataset Execution:** Train and evaluate strictly on the **Iris Binary dataset** (Dataset A).
- ☐ Implement class `RosenblattPerceptron(learning_rate=0.01, epochs=20, seed=42)`.
- ☐ Initialize $\mathbf{w}$ and b to zero tensors using `torch.zeros`.
- ☐ Implement sample-by-sample online iteration loop using `torch.dot(xi, self.weights) + self.bias`.
- ☐ Execute weight and bias updates immediately upon misclassification.
- ☐ Track misclassification counts per epoch in `self.errors_history`.
- ☐ Add early convergence termination when misclassifications reach zero.
- ☐ Implement `predict(X)` to output predicted labels on unseen test samples.

Problem 4: Multi-Class Two-Layer MLP from Scratch (Wine Multiclass Dataset)

Associated Concepts & Theory To model non-linear boundaries in multi-class problems, an intermediate latent representation is formed via a hidden layer. This architecture consists of an input projection, a Sigmoid non-linear hidden activation, and a Softmax output layer producing normalized class posterior probabilities. Optimization is driven by multi-class Cross-Entropy loss augmented with Frobenius norm L_2 weight regularization. All gradients are calculated analytically via matrix calculus backpropagation without using PyTorch autograd.

Mathematical Formulations Let $X \in \mathbb{R}^{N \times D_{\text{in}}}$, $W^{(1)} \in \mathbb{R}^{D_{\text{in}} \times D_{\text{hidden}}}$, $\mathbf{b}^{(1)} \in \mathbb{R}^{1 \times D_{\text{hidden}}}$, $W^{(2)} \in \mathbb{R}^{D_{\text{hidden}} \times D_{\text{out}}}$, and $\mathbf{b}^{(2)} \in \mathbb{R}^{1 \times D_{\text{out}}}$.

- **Hidden Layer Forward Pass:**

$$Z^{(1)} = XW^{(1)} + \mathbf{b}^{(1)}, \quad A^{(1)} = \sigma(Z^{(1)}) = \frac{1}{1 + \exp(-Z^{(1)})}$$

- **Output Logits & Numerically Stable Softmax:**

$$Z^{(2)} = A^{(1)}W^{(2)} + \mathbf{b}^{(2)}$$

$$\hat{Z}^{(2)}_{i,c} = Z^{(2)}_{i,c} - \max_k Z^{(2)}_{i,k}$$

$$A^{(2)}_{i,c} = \frac{\exp(\hat{Z}^{(2)}_{i,c})}{\sum_{k=1}^{D_{\text{out}}} \exp(\hat{Z}^{(2)}_{i,k})}$$

- **Regularized Cross-Entropy Objective:**

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \sum_{c=1}^{D_{\text{out}}} Y_{i,c} \ln(A^{(2)}_{i,c} + \epsilon) + \frac{\lambda}{2N} \left(\|W^{(1)}\|_F^2 + \|W^{(2)}\|_F^2 \right)$$

- **Analytical Backpropagation Equations:**

$$\delta^{(2)} = \frac{\partial \mathcal{L}}{\partial Z^{(2)}} = \frac{1}{N}(A^{(2)} - Y)$$

$$\frac{\partial \mathcal{L}}{\partial W^{(2)}} = (A^{(1)})^T \delta^{(2)} + \frac{\lambda}{N} W^{(2)}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(2)}} = \sum_{i=1}^{N} \delta^{(2)}_{i,:}$$

$$\delta^{(1)} = \frac{\partial \mathcal{L}}{\partial Z^{(1)}} = \left(\delta^{(2)} (W^{(2)})^T \right) \odot \left(A^{(1)} \odot (1 - A^{(1)}) \right)$$

$$\frac{\partial \mathcal{L}}{\partial W^{(1)}} = X^T \delta^{(1)} + \frac{\lambda}{N} W^{(1)}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(1)}} = \sum_{i=1}^{N} \delta^{(1)}_{i,:}$$

Tasks Checklist

- ☐ **Dataset Execution:** Train and evaluate strictly on the **Wine Multiclass dataset** (Dataset B).
- ☐ Implement class `TwoLayerMLP(n_features, n_hidden, n_classes, l2_reg=0.0, seed=42)`.
- ☐ Initialize weights using Xavier/Glorot normal distribution: $\sigma = \sqrt{2/(fan_in + fan_out)}$.
- ☐ Implement numerically stable static methods `sigmoid(z)` and `softmax(z)`.
- ☐ Implement `forward(X)` returning intermediate activations $Z^{(1)}, A^{(1)}, Z^{(2)}, A^{(2)}$.
- ☐ Implement `backward(X, A1, A2, Y_onehot)` returning analytical parameter gradients $[\nabla_{W^{(1)}}, \nabla_{\mathbf{b}^{(1)}}, \nabla_{W^{(2)}}, \nabla_{\mathbf{b}^{(2)}}]$.
- ☐ Implement manual SGD mini-batch iteration inside `fit_manual(X, y, lr, epochs, batch_size)`.
- ☐ Implement `predict(X)` returning predicted class integers via `torch.argmax(A2, dim=1)`.

Problem 5: Optimizer Integration via PyTorch API and Hyperparameter Grid Search (Wine Multiclass Dataset)

Associated Concepts & Theory First-order optimization methods dictate training efficiency and final generalization. Rather than writing optimizers by hand, standard PyTorch development couples custom forward/backward mathematical routines with production optimizer classes from `torch.optim`.

In this problem, you will attach PyTorch's native optimizer classes (`torch.optim.SGD` with momentum and `torch.optim.Adam`) to the manual tensor MLP parameters, load manual gradients directly into the `.grad` fields of the parameters, and run `optimizer.step()`. You will perform a multi-dimensional hyperparameter grid search across specified learning rates, momentum factors, and Adam moment coefficients on the Wine dataset.

Mathematical Formulations

- **SGD with Momentum (`torch.optim.SGD`):**

$$V_t = \beta V_{t-1} + g_t, \quad \theta_{t+1} = \theta_t - \eta V_t$$

  where β is the momentum factor and g_t is the analytical gradient.

- **Adam Optimizer (`torch.optim.Adam`):**

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \quad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

- **Hyperparameter Grid Search Spaces:**
  - Learning rates (3 values): $\eta \in \{0.005, 0.01, 0.05\}$
  - SGD Momentum factors (2 values): $\beta \in \{0.80, 0.95\}$
  - Adam β_1 values (2 values): $\beta_1 \in \{0.85, 0.95\}$
  - Adam β_2 values (2 values): $\beta_2 \in \{0.990, 0.999\}$

Tasks Checklist

☐ **Dataset Execution:** Execute all experiments on the **Wine Multiclass dataset** (Dataset B).

☐ Implement an updated training routine `train_with_torch_optimizer(model, optimizer, X_train, y_train, epochs, batch_size)`:

  ☐ Clear gradients using `optimizer.zero_grad()`.

  ☐ Compute analytical gradients via `model.backward()`.

  ☐ Assign each analytical gradient tensor directly to `param.grad` for each parameter in the model.

  ☐ Update parameters using `optimizer.step()`.

☐ **Part 1: SGD with Momentum Grid Search:**

  ☐ Evaluate Cartesian product of 3 learning rates × 2 momentum values (6 configurations).

  ☐ Instantiate `torch.optim.SGD(params, lr=lr, momentum=momentum)`.

  ☐ Log training loss and test accuracy for each run.

☐ **Part 2: Adam Optimizer Grid Search:**

  ☐ Evaluate Cartesian product of 3 learning rates × 2 β_1 values × 2 β_2 values (12 configurations).

  ☐ Instantiate `torch.optim.Adam(params, lr=lr, betas=(beta1, beta2))`.

  ☐ Log training loss and test accuracy for each run.

☐ Format and display summary tables comparing test accuracy across all parameter combinations.